

Debugging Python with dape

September 30, 2026

How the dape setup in `config.el` looks and works. The keys follow PyCharm, except where KDE takes `Ctrl+F1..F12` for switching desktops.

1 Keys

Action	PyCharm	Here
Start debugging	<code>Shift+F9</code>	<code>S-<f9></code> (then <code>debugpy</code>)
Resume	<code>F9</code>	<code><f9></code>
Step Over	<code>F8</code> <code>>~</code>	
Step Into	<code>F7</code>	<code><f7></code>
Step Out	<code>Shift+F8</code>	<code>S-<f8></code>
Toggle breakpoint	<code>Ctrl+F8</code> / click gutter	<code>C-S-b</code> / left-click fringe
Stop	<code>Ctrl+F2</code>	<code>S-<f5></code>
Conditional breakpoint	right-click breakpoint	middle-click fringe / <code>C-x C-a e</code>
Log breakpoint (no suspend)	<code>Shift</code> -click gutter	right-click fringe / <code>C-x C-a l</code>
Run to Cursor	<code>Alt+F9</code>	<code>C-x C-a u</code>
Evaluate Expression	<code>Alt+F8</code>	<code>C-x C-a x</code> , or type it in the REPL
Add to Watches	right-click → Add to Watches	<code>C-x C-a w</code> (symbol at point)
Rerun	<code>Ctrl+F5</code>	<code>C-x C-a r</code>
Pause	Pause button	<code>C-x C-a p</code>
Info panels (close/reopen)	Debug tool window	<code>C-x C-a i</code>
Debug console	Console tab	<code>C-x C-a R</code> (REPL)
Up/down the call stack	click in Frames	<code>C-x C-a <</code> / <code>C-x C-a ></code>
Remove all breakpoints	Remove All Breakpoints	<code>C-x C-a B</code>

`C-<f8>` and `C-<f2>` are bound too, but only reach Emacs if KDE’s “Switch to Desktop 8/2” shortcuts are freed. The `C-x C-a` prefix exists once dape has loaded, which happens when a Python file is opened. Not in dape: Smart Step Into, Mute Breakpoints, Force Step Into.

2 The screen while debugging

Treemacs	● 41 def step(x): ► 42 y = x * 2 x: 3 43 return y + 1			Claude
Outline	Stack (TAB v)	Scope (TAB v)	REPL	

- In the editor, ● in the fringe (the thin strip at the window's left edge, left of the line numbers) is a breakpoint, ► marks the line about to run, and the grey text after the current and previous lines shows the values of their variables.
- The terminal steps aside while a session runs and comes back on quit.
- Each panel's header line lists its tabs; TAB / S-TAB cycle through them, or click one. Left panel: Stack, Breakpoints, Threads, Modules, Sources. Middle panel: Scope, Watch.

3 A first session

1. Open a script and left-click the fringe next to a line inside a function or loop (or C-S-b on that line).
2. S-<f9>. The first time, type debugpy (TAB completes) and RET; it runs the current file. Afterwards the prompt comes pre-filled with the last command. To pass command-line arguments, add them before RET: debugpy :args ["--steps" "100"]. (debugpy-module runs the current directory as a package, like python -m.)
3. The program runs up to the breakpoint; Scope shows the local variables.
4. <f8> a few times to step over lines and watch the values change. <f7> on a function call steps into it; S-<f8> steps back out.
5. In Stack, move to a lower frame and RET: the editor jumps to the caller and Scope shows the caller's variables. Stepping always continues from the top frame.

6. <f9> continues to the next breakpoint.
7. S-<f5> quits: the panels close and the terminal comes back. If the program finishes by itself, the REPL says “Session terminated”; S-<f5> still closes the panels.

4 The panels

Panel	Key	What it does
Stack	RET	Select a frame: jump to its code and show its variables
Breakpoints	RET	Go to the breakpoint; on the exception rows (raised / uncaught) turn breaking on exceptions on or off
	D / d	Disable a breakpoint / delete it
	e	Edit a log breakpoint’s message
Scope	e	Expand or collapse a dict, list or object
	W	Add the variable to Watch
	=	Change the variable’s value while paused
Watch	W	Remove the expression from Watch
	C-x C-q	Edit the list of watch expressions as text

In Scope and Watch, RET does one of those three things depending on where the cursor is: on the + / - it expands, on the name it adds to Watch, on the value it edits. To watch a variable straight from the code, put the cursor on it and press C-x C-a w.

5 The REPL

- It shows everything the program prints. The program can’t read keyboard input (`input()`) in this setup.
- While paused, type any Python expression and it is evaluated in the selected frame: `len(sensors)`, `x * 2`, `vars(self)`. Assignments such as `x = 5` work too.
- One-letter commands: n step over, s step into, o step out, c continue, r restart, q quit. RET on an empty line repeats the last command, so n RET RET RET steps quickly.
- A variable named like a command (b c d e k m n o p q r s t u w) runs the command instead. Type a space before the name to see the variable, as in “ n”.

6 Breakpoints that don't just stop

- Conditional: middle-click the fringe (or `C-x C-a e`) and enter a condition such as `i == 50`. The program stops only when it is true.
- Log: right-click the fringe (or `C-x C-a l`) and enter a message such as `step {i}: x={x}`. The program doesn't stop; each hit prints the message in the REPL. Expressions go in `{}`.